electronics

MDPI

Article

Cognitive Agents Powered by Large Language Models for Agile Software Project Management

Konrad Cinkusz, Jarosław A. Chudziak and Ewa Niewiadomska-Szynkiewicz *

Faculty of Electronics and Information Technology, Warsaw University of Technology, 00-661 Warsaw, Poland; konrad.cinkusz.stud@pw.edu.pl (K.C.); jaroslaw.chudziak@pw.edu.pl (J.A.C.)
* Correspondence: ewa.szynkiewicz@pw.edu.pl

Abstract: This paper investigates the integration of cognitive agents powered by Large Language Models (LLMs) within the Scaled Agile Framework (SAFe) to reinforce software project management. By deploying virtual agents in simulated software environments, this study explores their potential to fulfill fundamental roles in IT project development, thereby optimizing project outcomes through intelligent automation. Particular emphasis is placed on the adaptability of these agents to Agile methodologies and their transformative impact on decision-making, problem-solving, and collaboration dynamics. The research leverages the CogniSim ecosystem, a platform designed to simulate real-world software engineering challenges, such as aligning technical capabilities with business objectives, managing interdependencies, and maintaining project agility. Through iterative simulations, cognitive agents demonstrate advanced capabilities in task delegation, inter-agent communication, and project lifecycle management. By employing natural language processing to facilitate meaningful dialogues, these agents emulate human roles and improve the efficiency and precision of Agile practices. Key findings from this investigation highlight the ability of LLM-powered cognitive agents to deliver measurable improvements in various metrics, including task completion times, quality of deliverables, and communication coherence. These agents exhibit scalability and adaptability, ensuring their applicability across diverse and complex project environments. This study underscores the potential of integrating LLM-powered agents into Agile project management frameworks as a means of advancing software engineering practices. This integration not only refines the execution of project management tasks but also sets the stage for a paradigm shift in how teams collaborate and address emerging challenges. By integrating the capabilities of artificial intelligence with the principles of Agile, the CogniSim framework establishes a foundation for more intelligent, efficient, and adaptable software development methodologies.

Keywords: cognitive agents; large language models; Agile software project management; scaled Agile framework; CogniSim; multi-agent systems

check for updates

Academic Editor: Ping-Feng Pai

Received: 20 November 2024
Revised: 21 December 2024
Accepted: 25 December 2024
Published: 28 December 2024

Citation: Cinkusz, K.; Chudziak, J.A.; Niewiadomska-Szynkiewicz, E. Cognitive Agents Powered by Large Language Models for Agile Software Project Management. *Electronics* **2025**, *14*, 87. https://doi.org/10.3390/electronics14010087

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Software engineering has undergone significant transformations due to rapid technological progress. The integration of sophisticated tools and methodologies is now essential to manage the increasing complexity and scale of modern software systems [1,2].

1.1. Background

In this rapidly changing landscape, the adoption of advanced technologies plays a critical role in addressing the challenges posed by large-scale and complex software systems.

Today's software development environment faces significant challenges, including managing extensive codebases, ensuring security, and maintaining quality across distributed teams. Traditional methodologies, such as the Waterfall model, have given way to Agile frameworks that emphasize iterative development, customer collaboration, and flexibility [3].

Despite the advantages of Agile methodologies, they have inherent limitations when scaling and managing large-scale, complex projects effectively [4]. Managing complex software systems requires breaking down the development process into structured activities. A common approach involves applying generic activities, communication, planning, modeling, construction, and deployment for each major product function [5].

The Scrum, a framework within Agile methodologies, addresses large-scale software system challenges through structured processes and defined roles. As shown in Figure 1, it organizes work via artifacts and scheduled meetings. The Product Backlog, managed by the Product Owner, aligns requirements with stakeholder goals. The Scrum team, led by the Scrum Master, plans a Sprint Backlog for a time-boxed iteration called a Sprint. Daily Scrums track progress, while the Sprint Review assesses deliverables, and the Sprint Retrospective identifies improvements [6]. This cycle ensures transparency, accountability, and steady progress toward a quality product.

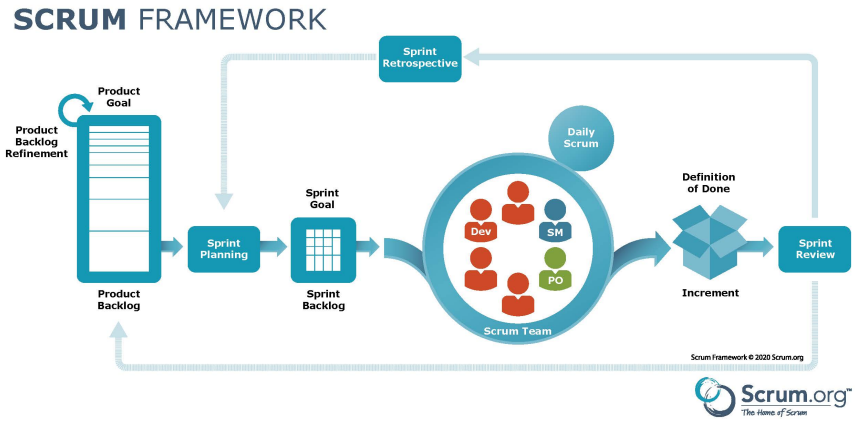

Figure 1. Scrum framework with key artifacts, meetings, and processes [7].

As detailed in Figure 2, software engineering is built on layered technology where each layer contributes to the quality and structure of the development process. This layered approach encompasses a focus on quality, processes, methods, and tools, collectively facilitating the systematic management of software engineering tasks [5]. It enables project teams to estimate resource requirements, schedule tasks, and define work products for each development stage.

Figure 2. Software engineering layers [5].

Multi-Agent Systems, comprising networks of autonomous agents that interact to achieve defined objectives within their environment, offer a resilient solution to these challenges [8]. These agents collaborate seamlessly, emulating human problem-solving processes and contributing to overall system efficiency [9].

Simultaneously, Large Language Models such as OpenAI's GPT-4, along with others like Google's PaLM and Meta's LLaMA, have transformed natural language processing by generating human-like content across various formats, including natural text generation for conversational tasks and storytelling, code generation to assist in software development, and tool use to perform complex workflows such as web searching, robotic operations, and online shopping [10]. In software engineering, LLMs can automate routine tasks like code completion, documentation, and debugging, reducing errors and leading to higher productivity [11–13].

Scaling Agile methodologies for large, complex projects introduces challenges such as coordinating across multiple teams, maintaining communication, and aligning organizational goals. Frameworks like SAFe, LeSS, and DAD address these issues by offering structured yet flexible approaches to Agile on scale, including defined roles, processes, and lifecycle models tailored to project needs. Although adoption requires overcoming cultural resistance and integration challenges, these frameworks have shown improvements in product quality, customer satisfaction, and delivery speed [13].

1.2. Related Work

Multi-Agent Systems (MASs) have been applied to distributed problems in dynamic environments [14], and the FIPA standards [15] established by the IEEE Computer Society enhance interoperability and efficiency within Agile frameworks. These standards streamline agent communication, coordination, and scalability. When combined with LLMs, as in frameworks like CoALA, MASs benefit from improved memory management, adaptive decision-making, and fluid inter-agent cooperation [16].

Recent advancements in MASs and LLMs are reshaping software engineering by introducing cognitive capabilities that improve reasoning, planning, and teamwork [17]. LLM-based MASs have emerged as a promising vision, equipped to address complex engineering tasks through autonomous problem-solving and scalable coordination [18]. Frameworks such as AGILE integrate reinforcement learning and LLMs, enabling agents to leverage tools and consult domain experts more effectively [19]. These agents have demonstrated applicability throughout the software lifecycle—from for requirements from engineering to maintenance—supported by specialized benchmarks and evaluation methods [20]. CodePori exemplifies the scalable integration of LLM-MAS, streamlining development tasks such as design, coding, and testing [21].

Over the past decade, Agile methodologies have significantly matured, with research underscoring the importance of theory-driven approaches to enhance their effectiveness [3]. Simultaneously, incorporating intelligent techniques, including machine learning and Bayesian networks, into Agile software development (ASD) has been shown to support decision-making, optimize effort estimation, and refine resource allocation [2]. Advances in MASs combined with LLMs illustrate how LLM-powered agents can promote more nuanced negotiation, role specialization, and collaborative decision-making [22]. Such systems can shift organizational dynamics from competition toward cooperative efforts, improving knowledge exchange and communication [8]. Furthermore, integrating theory-of-mind capabilities into LLM-based MASs promises richer context-sensitive interactions and deeper inter-agent comprehension [23].

1.3. Motivation and Research Gap

The combination of Multi-Agent Systems and Large Language Models creates a significant synergy, giving rise to cognitive multi-agent ecosystems that merge the strengths of both technologies [23,24]. This integration is particularly relevant in Agile software development, where flexibility, collaboration, and customer-centric approaches are paramount [1]. LLM-augmented MASs can facilitate more efficient task performance, adapting to dynamic

contexts and evolving requirements, and thereby more effectively address the complexities inherent in modern large-scale software engineering projects.

At the same time, frameworks for scaling Agile practices—such as the Scaled Agile Framework, Large-Scale Scrum (LeSS), or Disciplined Agile Delivery (DAD)—provide structured guidance for extending Agile principles across diverse and distributed teams [4,25], while these frameworks have achieved measurable improvements in areas like product quality and delivery speed, they still struggle to fully accommodate the heightened complexity and coordination demands of large-scale software initiatives. Existing approaches often lack the cognitive sophistication required to integrate advanced decision-making support and context-aware communication, leaving critical opportunities for more robust reasoning, planning, and adaptability unaddressed.

Although MASs have shown potential in distributed problem-solving and adaptive coordination [14,15], and LLMs have demonstrated effectiveness in tasks such as code generation, documentation, and interactive tool use [10–13], the literature lacks a cohesive framework that unifies these capabilities within a scaled Agile context. Recent work has explored cognitive MASs and LLMs individually [16–19], yet the integration of these technologies into established scaled Agile processes remains limited and fragmented.

Integrating cognitive agents and LLMs into frameworks like SAFe could provide deep, context-sensitive insights into project management and development workflows, enhancing not only the efficiency and quality of engineering tasks but also the agility with which teams respond to evolving objectives [26,27]. By explicitly addressing the current research gap—namely, the absence of a comprehensive, LLM-augmented MAS framework aligned with scaled Agile methodologies—this work aims to establish a foundation that can lead to improved coordination, decision-making, and adaptability in large-scale software engineering efforts.

1.4. Objectives and Problem Statements

This study aims to develop and analyze the CogniSim framework, a cognitive Multi-Agent System designed to transform software project management by integrating cognitive agents powered by LLMs. The primary objectives are to create a framework that automates routine project tasks, enhances workflows, and aligns with established Agile practices—particularly SAFe—to ensure scalability and effectiveness. By demonstrating its practical applications in software engineering, the framework will be evaluated through case studies and simulations.

Building upon the identified research gap and the outlined objectives for integrating LLM-augmented MASs within Scaled Agile Frameworks, this study's evaluation focuses on the following research questions (RQs):

1. RQ1: To what extent can cognitive agents, powered by LLMs, effectively simulate Agile roles and processes in a complex software development environment (e.g., SAFe)?
2. RQ2: How do variations in key parameters (e.g., model type, number of iterations, agent roles) influence the quality of outcomes, including code artifacts, documentation, and decision-making efficacy?

1.5. Approach and Methodology

This study adopts a design and implementation process to develop and refine the CogniSim framework as a problem-solving artifact. The methodology is aligned with Agile principles and SAFe guidelines, ensuring that the virtual environment and agent interactions reflect the complexities of large-scale software development [3,4,25]. By grounding the approach in Agile concepts and considering the evolving nature of software architectures,

we address the need for reconciling architectural documentation with Agile methodologies for better project outcomes [28].

Each cognitive agent represents a distinct Agile role—such as a Product Owner, System Architect, or QA Engineer—and operates autonomously to manage tasks, perform quality assurance, and provide continuous feedback. By doing so, the system emulates typical workflows, dependencies, and communication patterns in modern software engineering projects [1,2,5].

The technical foundation involves Python-based tooling, GPT-4 and GPT-3.5 language models, and the LangChain framework. LLM-powered agents process natural language instructions, generate documentation, and make context-aware decisions. This capacity allows the agents to adapt to evolving project goals, resource constraints, and shifts in priority, supporting Agile methodologies that emphasize responsiveness to change [11–13].

This methodology uses iterative simulations and case studies to evaluate the system's performance under various conditions. Simulated scenarios include activities such as Program Increment (PI) Planning, Iteration Execution, and Inspect-and-Adapt Workshops, all of which are core elements of SAFe-based Agile processes. These simulations enable controlled experimentation with different agent configurations, memory management strategies, and prompt designs to assess the system's effectiveness, efficiency, and scalability [17–19].

Evaluations rely on both qualitative and quantitative metrics. Key measures include the clarity and accuracy of generated artifacts, the extent to which decisions align with the defined architectural and business objectives, and the timeliness of deliverables. By comparing these outcomes against accepted best practices or human-generated baselines, the study identifies areas for improvement and informs subsequent adjustments to agent behavior, communication protocols, and underlying language model configurations [2,22]. This iterative feedback loop ensures that CogniSim remains adaptable as project complexity or organizational needs evolve [23,24].

1.6. Evaluation Framework

The evaluation applies both qualitative and quantitative techniques to assess how well the CogniSim framework supports Agile project management practices. Each simulation run produces a detailed record of interactions, decisions, and generated artifacts, as well as configuration parameters and anomalies encountered. This comprehensive documentation enables reproducibility and clear analysis pathways.

A qualitative thematic analysis is performed on the collected data to identify patterns that align with Agile principles, including communication efficiency, iterative refinement of tasks, and conformity with SAFe guidelines. Emergent themes are contrasted against established best practices to ensure that observed improvements are not coincidental. Quantitative metrics complement these qualitative insights, examining factors such as task completion times, clarity of generated outputs, and responsiveness to evolving requirements. These metrics provide a multifaceted perspective on performance. Although initial results suggest positive potential, future work involves extending the range of conditions tested, introducing more complex scenarios, and incorporating benchmarking against human-driven baselines to further refine the framework's utility and reliability.

1.7. An Outline of This Study

This study begins with Section 1, which frames the challenges of modern software engineering, emphasizing the necessity for scalable Agile practices and introducing Multi-Agent Systems and Large Language Models as transformative solutions. Next, Section 2 provides the theoretical foundation, covering Agile methodologies, cognitive agents, LLMs, and MAS concepts. Section 3 details its layered architecture, integrating LLM-powered

agents into Agile workflows. In the next Section 4, we illustrate its application through simulations, showcasing improvements in project communication, decision-making, and task execution. Then Section 5 discusses implementation specifics, highlighting modular design and Python-based tools for adaptability. It is followed by Section 6, which explores agent initialization, interaction protocols, and reproducibility in performance assessments. In Section 7, the framework's effectiveness is validated through metrics like task efficiency, deliverable quality, and Agile adaptability. Section 8 identifies opportunities for scaling, improving human–AI collaboration, and addressing ethical challenges. Finally, Section 9 underscores the CogniSim framework's contribution to advancing software engineering by merging LLM-powered agents with Agile methodologies.

2. Preliminaries

This section provides an overview of the fundamental concepts essential to this study, including agile software development methodologies with a focus on the Scaled Agile Framework, an introduction to cognitive agents and Large Language Models, and the fundamentals of Multi-Agent Systems and their applications in software engineering.

2.1. Agile Software Development

Agile software development is a group of methodologies that promote development, collaboration, and adaptability throughout the software development life cycle. Unlike the Waterfall model, Agile emphasizes customer collaboration, responses to change, and progressive delivery of tangible software [29].

One of the most adopted agile methodologies is Scrum, which structures development into time-boxed iterations called sprints. Each sprint results in a shippable product increment, allowing for consistent reassessment of project priorities and alignment with customer needs [30]. As illustrated in Figure 3, a generic Agile iteration cycle typically begins by selecting items from a prioritized backlog, proceeding through development and testing, demonstrating increments to stakeholders, and reflecting in retrospectives to guide continuous improvement.

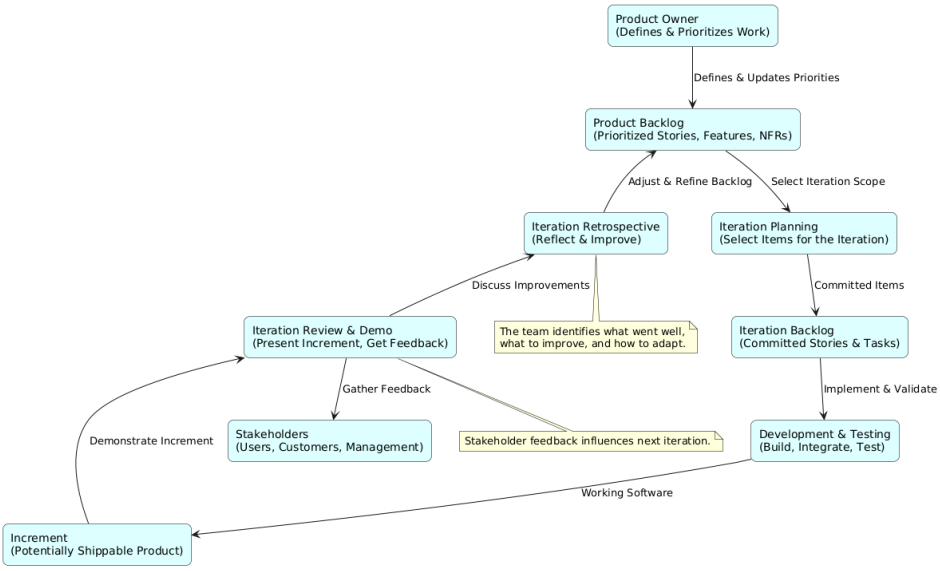

Figure 3. A generic agile iteration cycle illustrating planning, development, review, stakeholder feedback, and continuous improvement.

For expanding organizations dealing with advanced systems and multiple teams, the Scaled Agile Framework provides a coherent approach to scaling Agile practices [25]. SAFe

integrates principles from Lean thinking, Agile development, and systems thinking to facilitate coordination between teams and align development efforts with the goals of the organization. As illustrated in Section 8, the Scaled Agile Framework emphasizes five conceptual layers, Organizational Agility, Lean Portfolio Management, Enterprise Solution Delivery, Agile Product Delivery, and Team and Technical Agility, which collectively enable organizations to scale Agile practices and achieve business agility, with cognitive agents and MASs supporting each layer.

SAFe introduces the concept of the Agile Release Train (ART) [31], a structure that aligns multiple Agile teams and stakeholders to incrementally develop and deliver value within a value stream. While the ART is a SAFe-specific concept, the notion of multiple teams working in parallel and integrating their increments continuously is not exclusive to SAFe. Figure 4 offers a framework-neutral conceptual representation of how multiple Agile teams can plan, deliver, and integrate increments to support complex product development. This alignment facilitates improved coordination, synchronization, and collaboration between teams, enhancing the ability to handle advanced projects and product development.

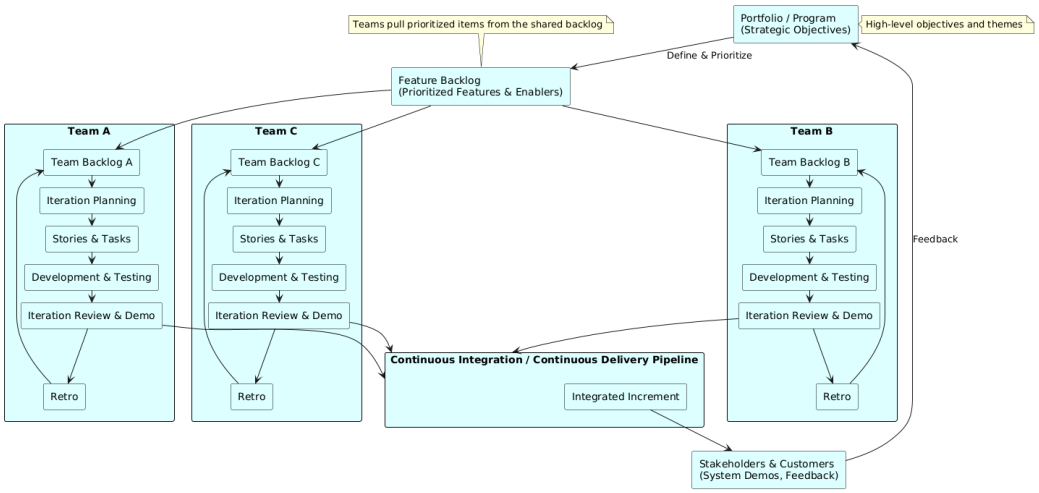

Figure 4. Conceptual scaled Agile iteration flow: multiple teams coordinating increments, integrating continuously, and aligning with strategic objectives.

Despite its benefits, the implementation of scaled Agile practices, including SAFe, presents challenges such as maintaining alignment between multiple teams, ensuring consistent communication, and integrating advanced technologies [32]. The integration of pioneering technologies such as cognitive agents and LLM can address these challenges by automating coordination tasks and enhancing communication efficiency.

2.2. Cognitive Agents and Large Language Models

Cognitive agents are smart systems capable of perceiving their environment, reasoning about inputs, learning from experiences, and taking actions to achieve goals [33]. They mimic human cognitive functions such as perception, learning, and problem-solving, enabling them to perform advanced tasks autonomously. A single cognitive agent, as shown in Figure 5, can range from a simple unit equipped with a specific function to a highly capable entity integrating diverse components such as memory, tools, and reasoning mechanisms. This flexibility allows cognitive agents to adapt to varying requirements, enabling modular scalability and dynamic task allocation based on evolving needs.

The development of Large Language Models, such as GPT-4, has advanced the capabilities of cognitive agents [34]. LLMs, such as GPT-4, are trained on diverse and extensive datasets, including sources like web pages, books, and scientific articles, enabling them to understand and generate language effectively [34]. This capability allows cognitive agents

to process input from natural language, participate in conversations, and perform tasks that require understanding context and semantics [35].

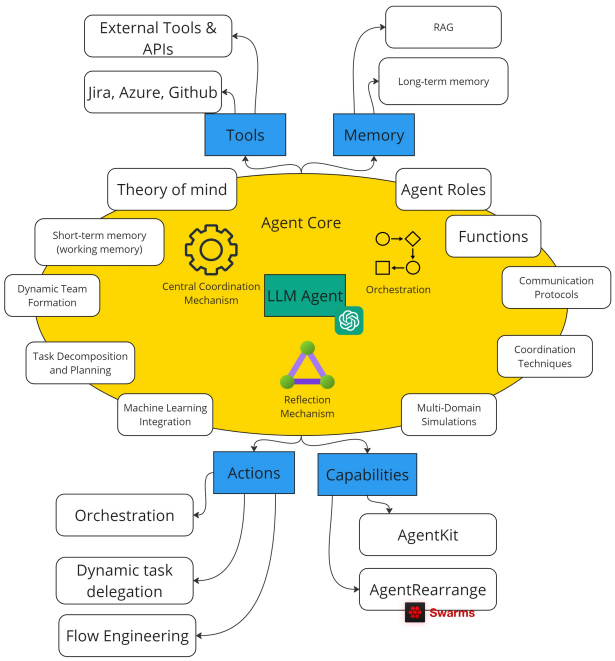

Figure 5. Single cognitive agent and its components [17].

In the context of software engineering, LLMs enhance cognitive agents by enabling them to automate code generation [36], assist in documentation [37], facilitate communication [23], and support decision-making [24]. By automating code generation, cognitive agents can produce code snippets or entire functions based on natural language descriptions, streamlining the development process. Assisting in documentation involves creating or updating documentation by summarizing code functionalities, ensuring that project documentation remains updated and substantial. Facilitating communication allows these agents to serve as virtual assistants in meetings, transcribing discussions and highlighting action items, thereby improving team collaboration. Additionally, supporting decision-making enables agents to analyze project data and provide insights that inform strategic decisions, enhancing the decision-making process within development teams [18].

The integration of LLMs into cognitive agents empowers them to handle tasks that require understanding advanced linguistic patterns and domain knowledge, making them valuable assets in Agile software development environments [18].

The integration of Large Language Models into cognitive agents enhances their linguistic capabilities while also advancing their overall cognitive functions through foundational components such as planning, memory, and tool use. In an LLM-powered autonomous system, as shown in Figure 6a, the LLM functions as the agent's central processor, coordinating processes to efficiently address complex tasks. Planning involves breaking down extensive objectives into manageable subgoals, facilitating systematic advancement through structured strategies. Reflection and self-assessment enable the agent to analyze prior actions, learn from mistakes, and optimize future strategies, thereby increasing the precision and effectiveness of its outcomes.

Memory is structured into short-term and long-term functionalities. Short-term memory employs in-context learning and prompt engineering to quickly assimilate new information, while long-term memory supports the storage and retrieval of extensive information over extended durations, often using external vector stores and rapid-access systems. Furthermore, the agent's ability to utilize tools enhances its capabilities beyond the limitations

of pretrained models. By integrating external APIs, real-time data, executable code, and proprietary information, the agent substantially improves its adaptability and operational efficiency in evolving environments.

Figure 6b illustrates the architecture of cognitive agents, highlighting the four interdependent components, perception, reasoning, learning, and action, arranged in a cyclic process. At the center of this architecture, Large Language Models act as the unifying core, enhancing each layer's functionality. The perception layer utilizes LLMs to improve natural language understanding, enabling agents to process and interpret complex linguistic inputs. In the reasoning layer, LLMs provide contextual insights and support advanced decision-making processes. The learning layer benefits from the continuous learning capabilities offered by LLMs, enabling agents to adapt and evolve using textual data. Finally, the action layer leverages LLMs to facilitate language-based task execution, ensuring efficient and accurate interaction with the environment. The cyclic design emphasizes the iterative and interconnected nature of these components, driven by the central role of LLMs.

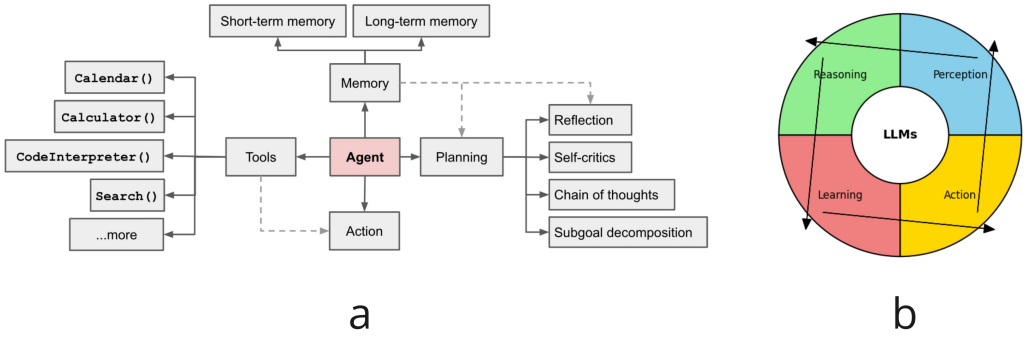

Figure 6. (**a**) Agent overview [38]; (**b**) cognitive agent architecture represented as a cyclic process with four components.

Recent advancements in the application of Large Language Models demonstrate their potential in educational and project management contexts, as evidenced by the MCQGen framework. This framework leverages LLMs combined with retrieval-augmented generation and advanced prompt engineering to automate personalized content creation [39].

2.3. Multi-Agent Systems

Multi-Agent Systems consist of interacting agents that collaborate to achieve individual or shared goals within an environment [40]. Each agent operates autonomously, perceiving its environment, making decisions, and taking actions. The agents in a MAS can be homogeneous—where all agents share identical capabilities, roles, and behavior—or heterogeneous, with agents differing in capabilities, roles, or objectives, allowing for specialization and complementary problem-solving. This distinction enables the MAS to be tailored to specific applications, with homogeneous systems offering simplicity and uniformity, while heterogeneous systems provide greater flexibility and complexity. In addition, their interactions can range from cooperative to competitive [41,42].

In software engineering, Multi-Agent Systems are applied in various domains. Figure 7 illustrates these applications, including distributed problem-solving, simulation and modeling, resource management, and collaborative software development.

In distributed problem-solving, agents divide complex problems into tasks and solve them concurrently, improving efficiency [43]. Simulation and modeling involve the use of MASs to replicate real-world systems, such as traffic networks or social behaviors, to analyze and predict outcomes [44]. Resource management tasks are managed by agents that handle resources in cloud computing environments, optimizing allocation and utiliza-

tion [45]. In collaborative software development, agents help integrate, test, and deploy code, thereby enhancing collaboration between development teams [46].

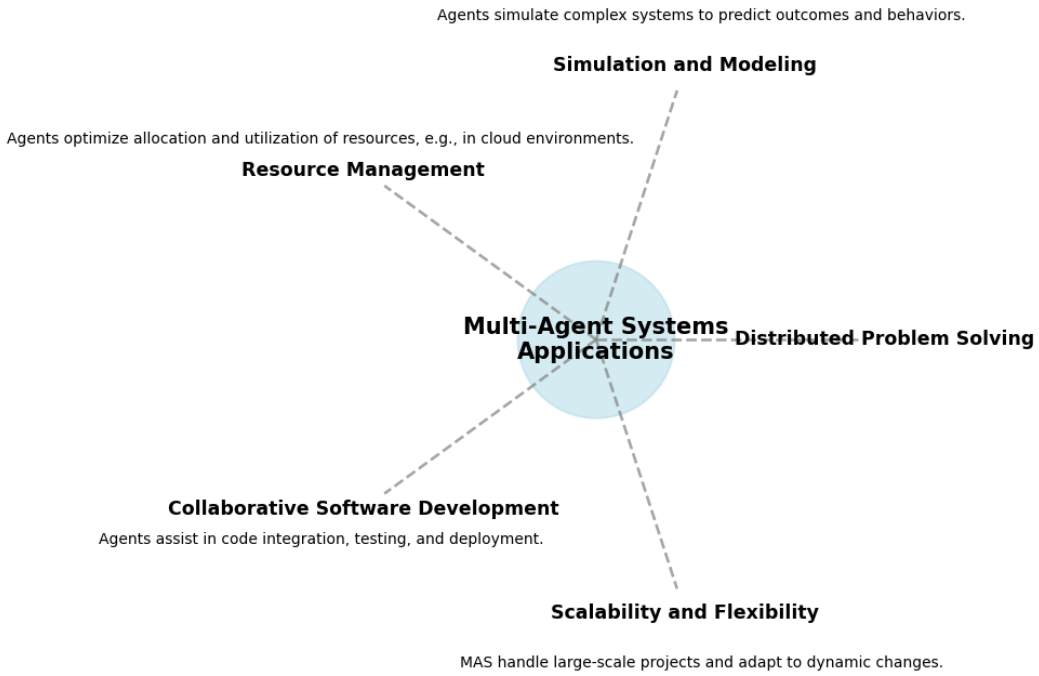

Figure 7. Applications of Multi-Agent Systems in software engineering.

The application of MASs in software engineering offers several benefits. Scalability is achieved because MASs can handle expanding systems by distributing tasks among agents, allowing the system to grow without a significant drop in performance [47]. Flexibility is provided by agents' ability to adapt to changes in the environment or requirements, making MASs suitable for systems where conditions frequently change [48]. Robustness is enhanced by the decentralized nature of the MAS, which reduces the single point of failure and increases the reliability of the system [49].

Frameworks such as ChatDev demonstrate the power of LLMs in unifying diverse roles through a structured, language-based communication model. By segmenting tasks into smaller, manageable phases and utilizing mechanisms like communicative dehallucination to ensure precision, these systems enhance the quality, completeness, and execution of generated software [50].

Integrating cognitive agents powered by LLMs into MASs combines the benefits of both technologies, leading to systems capable of advanced reasoning, learning, and collaboration [8]. This integration is particularly advantageous in Agile software development, where adaptability and enhanced communication are crucial.

3. CogniSim Framework

Before detailing its architecture, it is essential to clarify the nature of CogniSim and the capabilities it must encompass to address the challenges of large-scale Agile software development outlined in previous sections. CogniSim is a cognitive software framework designed to enhance and streamline Agile project management processes by integrating cognitive agents powered by Large Language Models into a Multi-Agent System environment. Rather than serving as a mere collection of standalone tools, CogniSim provides an intelligent system of LLM-augmented agents that collaborate with human teams. To effectively address previously discussed challenges, such as increasing complexity, scalability, and the need for efficient communication, CogniSim must incorporate several key features

and characteristics. These include cognitive assistance in understanding and reasoning about project contexts, the automation of labor intensive activities, inherent scalability aligned with Agile principles, and continuous quality assurance to ensure adherence to established practices. Table 1 summarizes these key attributes, highlighting how each directly addresses the challenges identified earlier.

Table 1. Key features and characteristics of the CogniSim framework.

Feature/Characteristic	Description and Role in Addressing Challenges
Cognitive assistance	LLM-powered agents provide human-like reasoning capabilities, enabling them to interpret natural language requirements and adapt to changing project contexts. This aligns with previous analyses highlighting the need for intelligent automation in complex software projects [23,34].
Automation of routine tasks	By automating coding, documentation, and backlog refinement, the framework reduces human workload and cognitive overhead, allowing team members to focus on strategic decisions. Prior studies show that LLM-based code generation and documentation support improve productivity [11,36].
Scalability and Agile alignment	CogniSim integrates seamlessly with Agile methodologies, particularly SAFe, ensuring synchronized development, continuous improvement, and effective communication across multiple teams and large-scale projects [4,25].
Quality assurance and methodology adherence	Specialized agents continuously verify that deliverables meet coding standards, adhere to Agile processes, and align with strategic objectives. This ensures high-quality outputs, as evidenced by the importance of formal quality metrics in software engineering [3,51].

With these foundational capabilities established, the following subsections detail the CogniSim framework's layered architecture, its agent categorization, its integration with SAFe, and the quality measures that guide its performance evaluation.

3.1. Framework Architecture

The CogniSim framework is built on a layered architecture that integrates cognitive agents, communication protocols, learning algorithms, decision-making frameworks, and collaboration tools. As depicted in Figure 8, the architecture consists of several key components.

The Foundation Layer is composed of a Large Language Model, such as GPT, BERT, and T5, which serves as the backbone for natural language understanding, reasoning, and task execution. These models provide the foundational capabilities necessary for enabling higher-layer functionalities. This layer allows the system to process, analyze, and generate human-like responses, making it suitable for integration with various applications.

The Multi-Agent System Layer consists of two distinct components. Layer 2 (A) represents the core MAS framework where agents interact, collaborate, and make autonomous decisions. These agents leverage the LLM foundation for reasoning and adaptiveness. Layer 2 (B) encompasses systems with AI integrations, such as Jira, Microsoft Azure, and GitHub, which provide operational support and external APIs for seamless task management, development, and communication.

The Cognitive Agents Layer (Layer 3) includes roles such as Product Owners, DevOps Engineers, and Development Teams. These agents manage and streamline tasks using integrated AI systems. Product Owners define and prioritize backlog items, ensuring alignment with business needs. DevOps Engineers maintain the CI/CD pipeline for smooth

deployments and operations. The Development Team implements features, conducts code reviews, and performs testing.

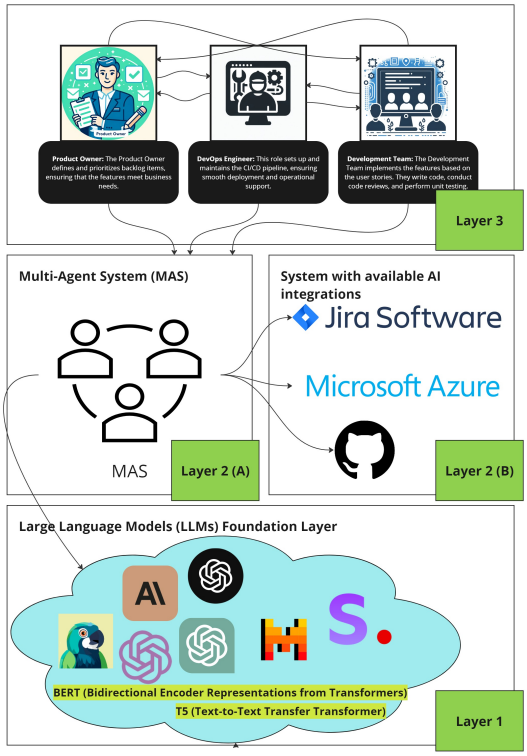

Figure 8. Layered ecosystem of CogniSim.

This modular and layered architecture ensures scalability, flexibility, and enhanced collaboration across human teams and AI-driven agents, optimizing software engineering workflows.

3.2. Agent Categorization

Within the CogniSim framework, agents are categorized based on their roles and responsibilities to optimize collaboration and task execution. The main categories include Manager Agents, Executor Agents, Quality Checker Agents, and Methodology Reviewer Agents. Manager Agents are responsible for high-level decision-making, resource allocation, and overseeing project progress [52]. These agents emulate roles such as Project Managers and Product Owners. Executor Agents focus on performing specific tasks such as coding, testing, and documentation [53], representing roles like Developers, QA Engineers, and Technical Writers. Quality Checker Agents ensure that deliverables meet predefined quality standards [54] by conducting code reviews, performing testing, and validating outputs against requirements. Methodology Reviewer Agents monitor adherence to Agile practices and methodologies [3], providing feedback on processes and suggesting improvements to enhance efficiency. Table 2 summarizes the agent categories and their primary functions within the framework.

Each agent is equipped with specific capabilities aligned with its role, enabling specialization and efficiency in task execution. The collaborative interaction among different agent types facilitates a comprehensive approach to project management, ensuring that all aspects of the software development lifecycle are effectively addressed.

Table 2. Agent categories and roles in CogniSim.

Agent Category	Primary Functions
Manager Agents	Decision-making, resource allocation, project oversight
Executor Agents	Task execution (coding, testing, documentation)
Quality Checker Agents	Quality assurance, code reviews, output validation
Methodology Reviewer Agents	Process monitoring, adherence to Agile practices, feedback provision

3.3. Integration with SAFe

The CogniSim framework aligns with the Scaled Agile Framework to enhance Agile practices in large-scale software development environments. The integration is achieved through several mechanisms. Firstly, cognitive agents are mapped to roles defined in SAFe, such as Release Train Engineers, Product Owners, and System Architects [25]. This role mapping ensures that agents fulfill responsibilities consistent with SAFe principles. Additionally, Manager Agents coordinate Executor and Quality Checker Agents to form virtual Agile Release Trains, facilitating synchronized development and delivery [31].

Methodology Reviewer Agents play a crucial role in promoting continuous improvement by monitoring processes and providing feedback, thereby fostering a culture of continuous improvement as advocated by SAFe [32]. Furthermore, agents focus on delivering value by aligning their tasks with the organization's strategic objectives, enhancing customer satisfaction [4]. Figure 9 illustrates how CogniSim integrates with the SAFe framework.

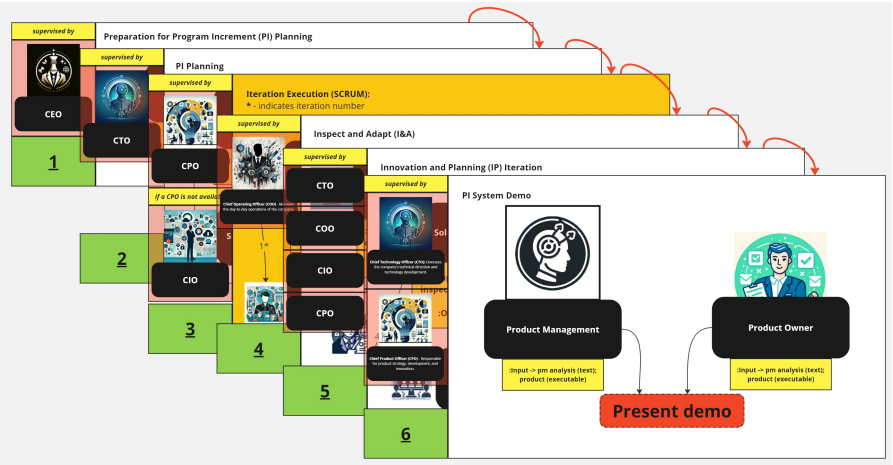

Figure 9. Integration of CogniSim with SAFe framework [52].

By integrating with SAFe, CogniSim enhances coordination across multiple teams, improves communication efficiency, and ensures that development efforts are aligned with organizational goals. The cognitive agents automate routine coordination tasks, allowing human team members to focus on strategic decision-making and innovation.

Cognitive agents dynamically adjust their reasoning and decision-making processes as project priorities evolve, integrating newly identified requirements and stakeholder feedback into their ongoing workflows. By continuously reassessing user stories, backlog items, and architectural constraints, these agents reconfigure tasks, timelines, and resource allocations in response to changing conditions. This iterative adaptation ensures that cognitive agents maintain momentum, alignment, and flexibility within Agile environments, effectively supporting teams as project contexts change.

4. Development Platform

The CogniSim framework operates on a structured and adaptable development platform that integrates cognitive agents and Large Language Models into a Multi-Agent System. This section examines the platform's architectural design, its essential components, and the implementation features that underscore its flexibility and scalability for Agile software development practices.

4.1. Platform Architecture

The CogniSim platform employs modular software engineering principles, ensuring efficient integration of cognitive agents and Large Language Models within a unified multi-agent framework. As depicted in Figure 10, the architectural approach prioritizes transparency, expandability, and system stability. The core directories, agents and agents_definitions, organize the agents' configurations, behaviors, and structural definitions. This logical segregation enhances the modularity of the system, facilitating effortless updates or enhancements to agent functionalities while preserving the integrity of the central system.

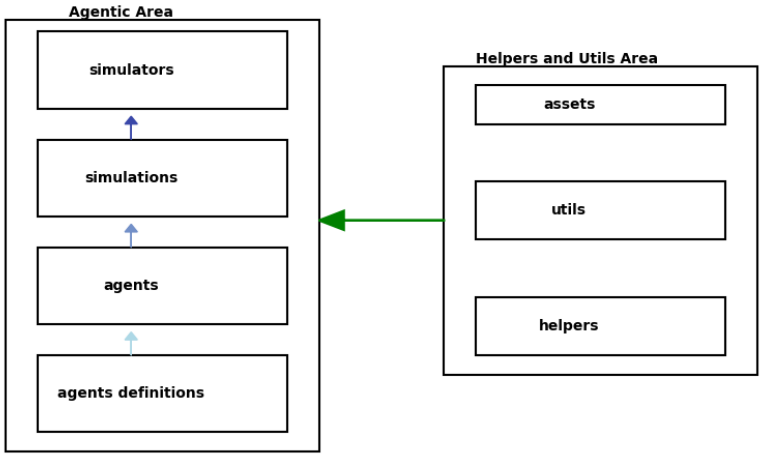

Figure 10. Project structure.

The simulations and simulators directories form the backbone of the experimental environment, enabling the simulation of various scenarios and tasks. These components replicate real-world dynamics and support iterative testing, allowing agents to participate in software project management tasks by adhering to Agile methodologies. The helpers and utils directories provide utility functions that support the main simulation processes, such as data analysis, inter-agent communication, and task orchestration, thereby enhancing the platform's flexibility and robustness.

The assets directory stores vital resources, including configuration files and datasets required for running the platform, ensuring that all necessary resources are readily available for simulation execution. Additionally, documentation files like README.md and requirements.txt offer comprehensive guidance on using the platform and managing dependencies, promoting clarity and reproducibility.

Overall, the architecture exemplifies an organized and scalable approach to developing cognitive agents capable of automating complex tasks in software project management. This modular design not only supports current simulations but also provides a solid foundation for future expansions, enabling more intricate simulations and a broader range of applications. By integrating advanced AI technologies and aligning with Agile methodologies, the CogniSim platform significantly enhances efficiency and productivity in software project management.

4.2. Key Components and Functionalities

The CogniSim platform is built using Python version 3.11.10, selected for its extensive ecosystem of libraries that are particularly well suited for artificial intelligence, LLMs, and data manipulation tasks [55]. The Integrated Development Environment utilized is Visual Studio Code (VS Code version 1.96.2), chosen for its versatility and strong community support, which facilitates the handling of complex simulations and Multi-Agent Systems efficiently.

A pivotal component of the platform is LangChain version 0.2, a framework designed to streamline the integration of LLMs into the Multi-Agent System. LangChain provides a modular and flexible environment that allows developers to combine various components, such as language models, prompts, memory modules, and external data sources, into cohesive workflows [56]. This framework is essential for managing the interactions between agents and leveraging the full potential of LLMs in generating context-aware and intelligent responses.

The platform also incorporates OpenAI's GPT-4 and GPT-3.5 models, selected for their advanced natural language processing capabilities, which are critical for enabling cognitive agents to perform high-level decision-making and communication [34]. Additionally, the platform leverages a variety of Python libraries, including tqdm for progress monitoring, pandas for data manipulation, websockets for real-time communication, and jsonschema for validating JSON structures, among others. These libraries collectively facilitate diverse functionalities such as monitoring simulation progress, managing structured data, enabling bidirectional communication between agents, and ensuring data integrity within the system.

4.3. Implementation Details

The CogniSim platform is designed with a modular and flexible architecture to support dynamic multi-agent simulations. At its core, the platform emphasizes abstraction and scalability, enabling users to configure and run simulations without requiring direct interaction with underlying code.

Agents in the platform are defined using a high-level configuration format, allowing their roles, behaviors, and interactions to be easily specified. This approach provides a clear separation between agent definitions and the core system, ensuring adaptability across diverse simulation scenarios. For example, roles such as Chief Technology Officer or Solution Architect can be incorporated seamlessly into simulations, demonstrating the platform's capacity for managing complex, role-specific interactions.

Simulations are orchestrated through a structured process that involves defining the simulation environment, setting parameters such as iteration limits and objectives, and ensuring all interactions are systematically recorded for analysis. The platform employs tools to track and document every simulation run, producing detailed logs and summaries for further evaluation. These outputs can be presented in user-friendly formats to aid in understanding the interactions and decision-making processes within the simulation.

5. Simulation Breakdown

This section provides an examination of the CogniSim framework's mechanics, focusing on the setup, agent interactions, and output generation processes. This substantial analysis ensures consistency and clarity in simulation execution while offering substantial data for evaluating the framework's effectiveness in Agile software project management.

5.1. Simulation Setup

The simulation environment within the CogniSim framework is configured to emulate concrete software development processes adhering to Agile methodologies. The setup begins with defining key simulation parameters, including the simulation name, maximum

iterations, and elements that guide the simulation's execution. These parameters establish the scope and boundaries of each simulation run, ensuring consistency and clarity in the simulation outcomes. Agents are initialized based on their roles, which are defined through JSON configuration files (Box 1) that outline their behaviors and interaction patterns. This initialization process is crucial for creating a tangible simulation environment where each agent operates according to its assigned responsibilities.

Box 1. JSON structure defining the Product Management Agent.

Agent Information

- **Agent Name:** Product Management Agent—Alex

Agent Definition

Prompt

- **Input Variables:**
  - client_analysis
  - solution_architect_feedback
  - instruction
- **Template Messages:**
  - **System:** As a Product Management professional, leverage your expertise in strategic planning and your deep understanding of market trends, customer needs, and the competitive landscape to develop a comprehensive vision and road map for the product. Your plan should align with the overall business strategy and customer expectations while being informed by the technical insights provided by the solution architect. Please adhere to the following steps:
    1. Analyze the detailed client analysis and solution architect feedback to inform your strategic direction.
    2. Define the product vision that aligns with the business strategy and customer needs.
    3. Develop a detailed road map and objectives for the upcoming Program Increment (PI), outlining the features and capabilities to be developed.
    4. Articulate the business context, including market changes and strategic goals, to justify the chosen direction.
    5. Engage with Product Owners and the solution architect to ensure alignment and address any technical or market-related feedback.
    6. Communicate the strategic plan to all stakeholders, ensuring clarity and buy-in.
    7. Establish metrics for success and outline how progress will be measured against the objectives.
  - **User:** This is the client analysis {client_analysis}.
  - **User:** This is the solution architect feedback {solution_architect_feedback}.
  - **Placeholder:** instruction
- **Template Format:** f-string

LLM

- **Model Name:** gpt-3.5-turbo
- **Temperature:** 0
- **Max Tokens:** null

Tools

- **Allowed Basic Tools:** None
- **Allowed External Tools:**
  - TavilySearchResults

Return Values

- **Output:**
  - output

Agent Type

- **Type:** dialogue_agent_with_tools

To provide further clarity, the JSON structure used for defining the Product Management Agent is included below. This example captures the essential parameters, including the prompt, input variables, roles, and allowed tools, which govern the agent's behavior and interactions within the simulation.

Environment variables are loaded to configure the simulation context, allowing agents to access necessary resources and data required for task execution. The JSON structure defines critical inputs, such as client analysis and solution architect feedback, which serve as starting points for simulations and provide agents with a clear understanding of project objectives and constraints. In addition, description files offer step-by-step task guides, delineating the sequence of interactions and ensuring alignment with Agile methodologies. This structured setup enables a controlled yet adaptable simulation environment, allowing agents to engage in complex interactions that mimic real-world project management scenarios.

5.2. Agent Interactions

In the simulation, agents interact through systematically organized communication protocols that facilitate structured dialogue and collaboration. The orderly progression of exchanges is maintained by a speaker selection mechanism, where agents employ their Large Language Models to produce contextually relevant responses that influence decision-making processes effectively.

Each agent maintains a record of past interactions, ongoing tasks, and project statuses, ensuring both continuity and adaptability. This functionality enables the CogniSim framework to replicate intricate interactions, offering a reliable environment for evaluating cognitive agents within the context of Agile project management.

5.3. Chat History Output

The Chat History Output Visualization (Boxs 2 and 3) provides a detailed view of the communication dynamics between cognitive agents during the simulation. This visualization highlights the flow of messages, enabling a closer analysis of agent interactions, decision-making processes, and collaborative behavior. Messages are presented with attribution to individual agents, truncated to maintain readability, and formatted to ensure clarity. By reviewing this output, readers can better understand the practical implementation of dialogue-driven simulations and how agents engage in real-time problem-solving within the Agile framework.

Box 2. ChatHistory—Iteration Execution.

UX Designers: During the Iteration Execution phase, UX Designers collaborate with the Development Team to refine designs, adjust specifications, and iterate on assets based on feedback and technical constraints.

DevOps Engineer: As the DevOps Engineer, ensure CI/CD pipelines support integration, testing, delivery, and automation to enhance the efficiency of builds, deployments, and troubleshooting processes with Development Teams.

System Team: The System Team supports integration and testing environments, resolves infrastructure issues, and collaborates with DevOps Engineers to maintain stability and address changes during Iteration Execution.

Development Team: The Development Team focuses on user stories, communicates design needs with UX Designers, ensures environment readiness with System Team, and raises dependencies or issues during the iteration.

Box 3. Chat History—preparing for program increment.

Product Management—Alex: System Architect, can you outline a high-level approach for using a Bayesian network to improve our real-time trading decisions on Coinbase?

System Architect—John: Great, let us dive a bit deeper into each step. Starting with the first one, Data Gathering, what kind of historical data are we talking about? How far back should we go? Furthermore, how do we ensure the real-time data are accurate and timely for making predictions?

System Architect—John: That makes sense. Now, looking at the second step, could you explain further how we define the structure and variables of the Bayesian network? What factors should we consider when deciding on the structure? Furthermore, how do we determine the relevant variables?

System Architect—John: I see, so the network structure is dynamic and might need adjustments over time. Moving on to the third step, Learning Parameters, could you elaborate a bit more on how we learn these parameters? What techniques or algorithms could we use? Furthermore, how do we validate the accuracy of these parameters?

System Architect—John: Thank you for explaining that. Now let us discuss the fourth step, Inference. Could you explain how this process works in detail? How do we use the learned parameters to make predictions? Furthermore, how do we handle uncertainty in the prediction process?

Product Management—Alex: Thank you for all the details provided so far. Now that we have refined the approach, could you please provide a final consolidated list of the key tasks, roles, and next steps for implementing this Bayesian network integration?

System Architect—John: Absolutely. The final plan for implementing the Bayesian network for real-time crypto trading on Coinbase is as follows: (. . .)

5.4. Output Analysis

The outputs generated by the simulations provide substantial data for evaluation and analysis. Each simulation run produces chat logs that capture the full interaction between agents, documenting the flow of communication, decision-making processes, and task executions. These logs are saved in both HTML and text formats, offering different levels of accessibility and readability for various analytical purposes.

HTML files present the chat history in a structured and visually clear format, with color-coded messages and agent-specific icons enhancing the clarity of interactions. This visualization aids in quickly identifying patterns, key decision points, and the effectiveness of communication among agents. Text files provide a straightforward and unformatted record of the dialogues, suitable for in-depth textual analysis and comparison against predefined benchmarks.

Execution logs capture the simulation's runtime details, including timestamps, system interactions, and any warnings or errors encountered during execution. These logs are essential for debugging purposes and for understanding the underlying processes that drive agent interactions. Additionally, configuration files documenting the input parameters and agent setups are generated, ensuring that each simulation run is reproducible and that the conditions under which the simulation was conducted are well documented.

The analysis of these outputs involves assessing the performance of agents based on predefined metrics such as task completion time, quality of deliverables, and communication efficiency. By evaluating these metrics, insights can be gained into the strengths and limitations of the CogniSim framework, as well as the effectiveness of LLM-powered cognitive agents in managing Agile software development tasks. This substantial output analysis is critical for validating the framework's capabilities and for identifying areas for further improvement and optimization.

5.4.1. Visualization of Results

The HTML output files generated by the simulations provide a visual representation of agent interactions, which is essential in assessing the flow and coherence of communication. Each agent's messages are color-coded and accompanied by icons, allowing for quick differentiation and analysis of individual contributions within the dialogue. This visual

format facilitates the identification of communication patterns, highlighting how agents coordinate tasks, resolve conflicts, and make collective decisions. By reviewing these visual logs, researchers can gain a deep understanding of the agents' collaborative dynamics and the overall effectiveness of the simulation in replicating concrete Agile interactions.

5.4.2. Log Analysis

Execution logs play a pivotal role in the simulation breakdown by offering a complete trace of the simulation's execution flow. These logs include timestamps, system actions, and any warnings or errors that occur, providing significant insights into the functional aspects of the simulation. For example, deprecation warnings and other system-level messages can indicate potential issues with the codebase or the need for updates to dependencies. Analyzing these logs helps in identifying and troubleshooting technical problems, ensuring the robustness and reliability of the simulation environment. Moreover, the logs capture the sequence of agent actions and responses, enabling a thorough examination of the agents' decision-making processes and their adherence to the simulation's objectives.

5.4.3. Reproducibility

Ensuring reproducibility is a fundamental aspect of the CogniSim framework, achieved by completely documenting all input parameters and agent configurations used in each simulation run. Configuration files in JSON format store full information about agent roles, behaviors, and interaction patterns, allowing simulations to be replicated accurately under identical conditions. This comprehensive documentation enables researchers to validate results, conduct repeated experiments, and compare outcomes across different simulation runs. Reproducibility is essential for establishing the reliability of the framework and for facilitating continuous improvement based on consistent and comparable data.

Overall, the simulation breakdown provides a substantial overview of how the CogniSim framework operates, detailing the setup, agent interactions, and output generation processes. This analysis ensures that each simulation run is executed in a controlled and consistent manner, providing significant data for evaluating the framework's effectiveness in enhancing Agile software project management through cognitive agents powered by Large Language Models.

6. Case Study

This case study aims to address this specific research question: "How effectively can cognitive agents, powered by LLMs, replicate and enhance key roles and processes within Agile (SAFe) software development environments?" While this new question complements and builds upon the overarching research questions (RQ1 and RQ2) introduced earlier, our immediate focus here is on evaluating the CogniSim framework against defined quality characteristics—namely, decision-making effectiveness, communication efficiency, and adaptability under realistic project constraints. The findings from this chapter will serve as groundwork for a more detailed reflection on RQ1 and RQ2, which will be revisited and discussed at length in the next chapter.

6.1. Case Study Overview

The simulation focuses on a comprehensive software project that involves the integration of advanced APIs and the development of comprehensive business logic. The roles simulated within this environment are described in Section 3. By modeling these interactions, the case study aims to showcase how the CogniSim framework can effectively handle real-world software engineering challenges, such as aligning technical capabilities with business objectives, managing dependencies, and maintaining project agility.

6.2. Case Study Approach

The methodology for this case study employs a qualitative research approach, utilizing the CogniSim framework to simulate interactions among various astute agents. The process begins with the simulation setup, where the project scope, objectives, and agent roles are meticulously defined based on a realistic software development scenario. Agents are then configured using JSON files that specify their roles, behaviors, and responsibilities, ensuring a high level of customization and scalability. Data collection occurs over multiple simulation runs corresponding to distinct SAFe phases. In each run, we log agent communications, decisions, and outcomes (e.g., features implemented, technical decisions made). We capture the following:

- Agent dialogue and decisions: All agent-to-agent and agent-to-environment messages are recorded, providing a complete trace of negotiation, planning, and execution activities.
- Performance metrics: We measure task completion times and adherence to project timelines.

Following the configuration, the simulation is executed through multiple iterations to emulate the cyclical nature of Agile processes, including Program Increment (PI) Planning, Iteration Execution, and Retrospectives. During each iteration, data are collected on agent interactions, task completions, and decision-making processes. These data are then analyzed to evaluate the performance of the agents against predefined metrics such as task completion time, quality of deliverables, and communication efficiency.

The simulation workflow is illustrated in Figure 11, which shows the iterative Agile process from setup through data analysis.

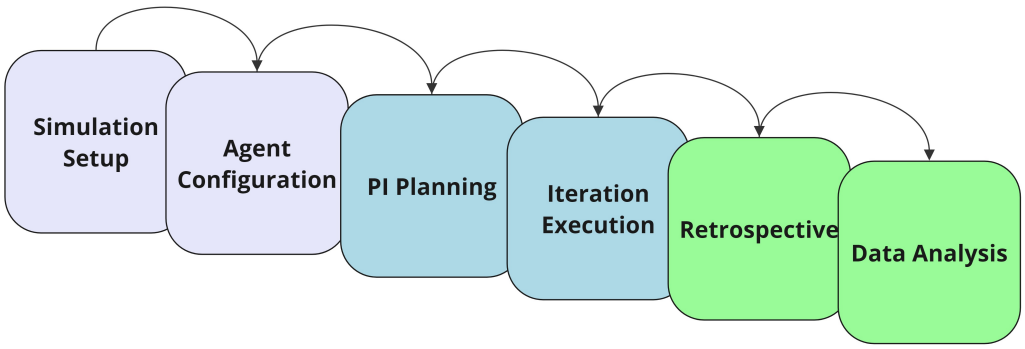

Figure 11. Simulation workflow in CogniSim, showing the iterative Agile process from setup through data analysis.

6.3. Results and Analysis

The simulation results provide compelling evidence of the CogniSim framework's effectiveness in enhancing Agile software development processes. One of the most significant findings is the enhancement in decision-making capabilities of the agents. Astute agents demonstrated the ability to analyze advanced scenarios and make informed decisions swiftly, leading to more efficient problem-solving and task management. This was particularly evident in the PI Planning phase, where agents successfully negotiated feature priorities and identified technical dependencies, ensuring that project objectives were aligned with both business goals and technical feasibility.

Efficiency improvements were another significant outcome, with task completion times being significantly reduced compared to human-managed processes. The agents' ability to process information rapidly and execute tasks without the typical bottlenecks associated with human teams contributed to this increased efficiency. Additionally, the

consistency and reliability of the agents' outputs were enhanced, as they maintained a high level of quality and minimized errors throughout the simulation.

Agents continued to communicate effectively, and the distribution of tasks across multiple agents reduced the load on individual components without increasing coordination overhead. Although these results are preliminary and gathered from controlled simulations, they indicate that the CogniSim framework can grow in complexity and team size without a corresponding drop in performance.

Overall, the results indicate that the CogniSim framework can effectively augment Agile software development practices by automating routine tasks, enhancing decision-making processes, and fostering efficient collaboration among team members. These improvements contribute to a more streamlined and productive software development lifecycle, demonstrating the potential of integrating astute agents powered by LLMs into project management scenarios.

6.4. Key Insights and Applications

The case study emphasizes several important insights regarding the integration of perceptive agents in Agile software project management. Firstly, the scalability of the CogniSim framework is notable, as it supports larger teams and advanced projects without a corresponding increase in resource requirements. Specifically, by incrementally increasing both the number of agents and project complexity, we observed stable task completion rates, manageable communication overhead, and consistent decision-making quality, thereby providing preliminary evidence for scalability.

Secondly, the adaptability of the agents proves essential in dynamic project environments. Their capacity to address shifting requirements and evolving project conditions ensures that the development process remains agile and attuned to stakeholder needs. This adaptability is particularly advantageous in circumstances requiring swift responses to unexpected challenges or the incorporation of new features.

Lastly, the collaboration between human team members and perceptive agents underscores the synergistic strengths of both; while agents efficiently automate routine tasks and manage large volumes of data, human oversight is indispensable for strategic decision-making and addressing complex issues that demand contextual understanding and innovative problem-solving. This effective partnership between humans and perceptive agents enhances project management by leveraging their combined strengths to achieve superior outcomes in software development.

In summary, through a structured qualitative methodology, defined research questions, thematic data analysis, illustrative dialogue snippets, and performance measurements, this case study provides a more rigorous demonstration of the CogniSim framework's capabilities. Future work will involve more extensive quantitative analyses and real-world field studies to further validate these findings.

7. Experiments and Results

Building upon the research questions outlined in Section 1, this section evaluates the CogniSim framework's effectiveness in achieving the previously stated objectives. Rather than restating those questions here, we focus on how well the framework addresses the identified challenges and objectives presented in the introduction.

To address these questions, we focus on evaluating specific characteristics such as performance, quality of deliverables, and collaborative effectiveness. Performance is measured through metrics like task completion time and backlog reduction, reflecting the efficiency of the system. Quality of deliverables considers factors such as adherence to coding standards and the clarity of generated documentation, emphasizing the reliability

and usability of outputs. Collaborative effectiveness assesses aspects like communication efficiency and adaptability, highlighting the system's ability to facilitate teamwork and respond to dynamic scenarios. These attributes act as dependent variables, while independent variables include the choice of LLM model (e.g., GPT-3.5-turbo versus GPT-4), the number of simulation iterations, agent role configurations, and prompt parameters such as temperature and memory settings (as shown at Box 4). By examining these variables, we aim to analyze the interplay between system inputs and outcomes.

Box 4. JSON structure for simulation configuration.

Simulation Configuration

- **Simulations:**
  - Simulation 1:
    - **sim_id:** 1
    - **model_type:** gpt-3.5-turbo
    - **iterations:** 10
    - **temperature:** 0.7
    - **agents_involved:**
      - Product Management
      - System Architect
  - Simulation 2:
    - **sim_id:** 2
    - **model_type:** gpt-4
    - **iterations:** 10
    - **temperature:** 0.7
    - **agents_involved:**
      - Product Management
      - System Architect

In this experimental setup, the cognitive agents collectively simulate the development of a hypothetical enterprise-level software solution—such as a web-based application integrating external payment APIs, user management modules, and advanced data analytics features. As a result, agents produce various software artifacts: code snippets that implement API integrations, user stories refined into actionable backlog items, architectural decision logs, UI/UX design recommendations, and deployment pipeline configurations. By analyzing these outputs, we gain insight into the quality and completeness of the software engineering process facilitated by CogniSim.

7.1. Experimental Design

The experiments conducted within the CogniSim framework were designed to evaluate the performance and effectiveness of cognitive agents in simulating Agile software development processes. These simulations replicate different SAFe phases—Preparation for Program Increment (PI) Planning, PI Planning, Iteration Execution, Inspect-and-Adapt Workshop, Innovation and Planning (IP) Iteration, and PI System Demo—thereby providing multiple data points to assess how agents behave under varying conditions and project stages.

The independent variables manipulated during these experiments include the following:

- Model type (independent variable): GPT-3.5-turbo or GPT-4.
- Number of iterations (independent variable): Variable between runs (e.g., 10, 50) to observe long-term behavior.
- Agent roles (independent variable): Adjusting which roles are included (Product Management, System Architect, Development Team, etc.).

- Temperature and prompt settings (independent variable): Influencing the creativity and precision of agent outputs.

The dependent variables include the following:

- Performance metrics: Task completion time, backlog reduction rate.
- Quality metrics: Code adherence to standards, clarity of documentation, correctness of architectural decisions.
- Collaboration and communication metrics: Frequency and quality of agent interactions, consistency in decision-making, adaptability to changing requirements.

The experimental design encompassed six cohesive simulations, each representing a pivotal SAFe phase, assigning roles to agents such as Product Managers, System Architects, Development Teams, UX Designers, and DevOps Engineers. By running these simulations through multiple iterations, the experiments aimed to capture the cyclical nature of Agile processes, allowing for a cohesive assessment of agent interactions, task management, and decision-making efficacy.

Agent Parameters for Experimental Variations

To comprehensively assess the impact of pivotal factors on the performance of cognitive agents, a range of agent parameters were systematically varied across simulation runs. Table 3 outlines the key agent parameters adjusted during the experiments.

Table 3. Agent parameters.

Parameter	Description	Possible Values
Model type	The language model used for agent responses.	GPT-3.5-turbo, GPT-4.
Number of iterations	Number of turns or exchanges between agents.	Any positive integer (e.g., 10, 50, 100).
Temperature	Controls randomness in the model's output (creativity level).	0.0 (deterministic) to 1.0 (maximum randomness) [57].
Max tokens	Maximum length of responses from agents.	Any positive integer (e.g., 150, 500).
Agent roles	Different roles or agents involved in the simulation.	Product Management, System Architect, Development Team, QA Engineer, etc.
Prompt templates	Different initial prompts or instructions for agents.	Varied prompts per agent to test impact on responses.
Input variables	Specific input data provided to agents (e.g., client analysis, objectives).	Different scenarios or datasets for testing.
Selection function	Method for selecting the next speaker in the simulation.	Alternate speakers, random selection, directed selection.
Elaboration functions	Use of functions that elaborate or expand on topics (e.g., topic elaboration).	Enabled, disabled.
API parameters	Other OpenAI API parameters like presence_penalty, frequency_penalty.	presence_penalty: −2.0 to 2.0; frequency_penalty: −2.0 to 2.0.
Agent memory	Amount of prior conversation history agents remember.	Full memory, limited memory (e.g., last 3 messages).
Agent personality or style	Communication style of agents (affects language used).	Formal, casual, technical, persuasive.

An example setup for simulations with varied simulation parameters is presented in Table 4.

Table 4. Example simulations with varied parameters.

Sim ID	Model Type	Iterations	Temperature	Agents Involved	Notes
1	GPT-3.5-turbo	10	0.7	Product Management, System Architect	Baseline simulation
2	GPT-4	10	0.7	Product Management, System Architect	Testing with GPT-4
3	GPT-3.5-turbo	50	0.7	Product Management, System Architect	Increased iterations
4	GPT-3.5-turbo	10	0.5	Product Management, System Architect	Lower temperature (less randomness)
5	GPT-3.5-turbo	10	0.9	Product Management, System Architect	Higher temperature (more randomness)
6	GPT-3.5-turbo	10	0.7	Product Management, System Architect, Dev Team	Added Development Team agent

By varying these parameters, this study aimed to investigate their effects on the agents' performance, interaction dynamics, and the overall quality of the simulations. The systematic variation of parameters allowed exploration of different scenarios and configurations, providing insights into optimal settings for cognitive agent simulations within the Agile framework.

7.2. Results and Analysis

The results of the experiments are captured in Table 5, detailing the performance metrics obtained during the simulations. The table includes key metrics such as unique content percentage, diversity score, completion score, and sentiment stability for each simulation run.

Table 5. Simulation results.

Sim ID	Model Type	Unique Content	Diversity Score	Completion Score	Sentiment Stability
1	gpt-3.5-turbo	100.00	0.45	50.00	0.00
2	gpt-4	100.00	0.77	0.00	100.00
3	gpt-3.5-turbo	83.67	0.43	50.00	100.00
4	gpt-3.5-turbo	11.11	0.46	0.00	100.00
5	gpt-3.5-turbo	33.33	0.46	0.00	100.00
6	gpt-3.5-turbo	100.00	0.63	50.00	33.33

To provide a comprehensive overview, the graphical representation in Figure 12 illustrates key performance metrics derived from the simulations:

- Unique content percentage: A bar chart highlights the proportion of unique content generated in each simulation, reflecting the level of creative and non-redundant output.
- Diversity score: A line graph presents the diversity scores across simulations, emphasizing variations in the breadth and inclusivity of content.

- Completion score and sentiment stability: A combined plot showcases completion rates and sentiment stability trends, illustrating the balance between task execution and emotional consistency.
- Radar chart of average metrics: A radar chart summarizes the overall performance metrics, including unique content percentage, diversity score, completion score, context retention, and sentiment stability, offering an integrated view of agent performance.

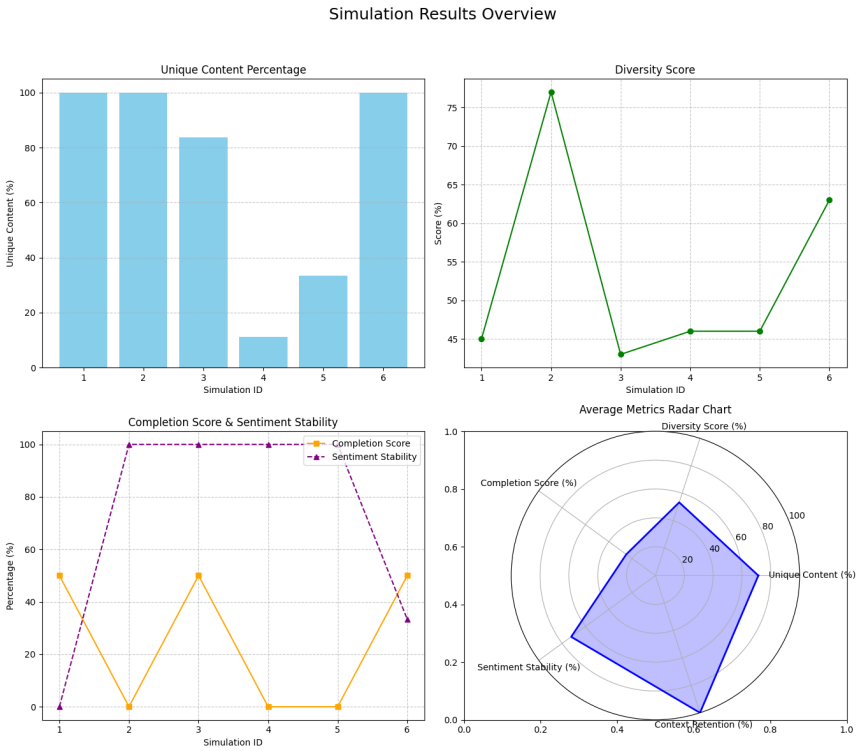

Figure 12. Simulation results.

These visualizations offer an intuitive understanding of the simulation outcomes, enabling clear identification of strengths and areas for improvement. For instance, the unique content bar chart highlights simulations with high redundancy, while the radar chart underscores the overall balance among key performance indicators.

7.3. Quality Measures and Performance Metrics

To evaluate the effectiveness of the CogniSim framework, a set of quantitative and qualitative measures was used, capturing both the efficiency of cognitive agents and their adherence to best practices. These metrics included Task Completion Time, Quality of Deliverables, Communication Efficiency, Resource Utilization, Adaptability, and Compliance with Agile Practices. Task Completion Time measures the duration taken by agents to complete assigned tasks, providing insights into their efficiency and speed [58]. Quality of deliverables is assessed through code quality metrics such as cyclomatic complexity, code coverage, and adherence to coding standards [51]. Communication efficiency is evaluated based on the frequency and clarity of interactions between agents and between agents and members of the human team, indicating the effectiveness of collaboration [59]. Resource utilization monitors how effectively agents allocate and manage resources, reflecting optimization capabilities [53]. Adaptability gauges agents' responsiveness to evolving project requirements and environmental conditions [48], while compliance with Agile Practices ensures alignment with established Agile and SAFe methodologies, maintaining industry best practices [32]. These comprehensive metrics facilitate a multidimensional evaluation of agent performance. Continuous monitoring and analysis enable the identification of areas

for improvement, guiding iterative enhancement of the CogniSim ecosystem. Furthermore, analyzing the impact of varying simulation parameters, such as model type, temperature settings, number of iterations, and agent roles, provides deeper insight into how these factors influence the metrics. For example, adjusting the temperature parameter affects the variability of agent responses and, in turn, influences Communication Efficiency and Quality of Deliverables. Increasing the number of iterations allows observation of agent behavior over extended interactions, offering further understanding of their Adaptability and Compliance with Agile Practices. By systematically refining these parameters, optimal configurations can be identified to enhance overall system performance.

7.4. Key Findings and Implications

The experiments reveal that cognitive agents can effectively handle structured Agile phases (e.g., PI Planning, Iteration Execution) by producing relevant user stories, code snippets aligned with predefined requirements, and infrastructure scripts for CI/CD pipelines. For example, during Iteration Execution, agents delivered code integrating a payment API and generated corresponding test cases, thereby demonstrating the creation of tangible software artifacts.

However, the agents' effectiveness was mixed in more open-ended phases (e.g., Inspect and Adapt), highlighting challenges in adaptability and innovation without further prompt engineering. The choice of model type (GPT-4 vs. GPT-3.5-turbo) and parameter settings influenced the quality, clarity, and timeliness of outputs. While GPT-4 excelled in generating well-structured code and documentation, lower temperatures enhanced precision and reliability.

In summary, these experiments answer our research questions (RQ1 and RQ2) by showing that cognitive agents can simulate various Agile roles effectively, produce meaningful software artifacts, and maintain performance quality under controlled experimental conditions. Adjusting independent variables (such as model type and iteration length) allowed us to identify settings that optimize dependent variables (such as code quality or responsiveness to changing requirements). Future work will focus on refining these experimental methods, incorporating more diverse project types, and validating results with real-world developer feedback.

8. Future Work

The future directions of this study encompass multiple pivotal areas aimed at enhancing the integration of cognitive agents, Large Language Models, and Multi-Agent Systems within Agile development frameworks. These directions focus on research extensions, technological advancements, and the addressing of ethical concerns, ensuring the continuous evolution and practical applicability of the proposed framework.

8.1. Research Extensions

The integration of cognitive agents, Large Language Models, and Multi-Agent Systems within Agile development frameworks offers substantial opportunities for further research. A pivotal area for extending the current study involves exploring the scalability and interoperability of the LLM ecosystem across software projects of different sizes and complexities. This entails assessing the ecosystem's performance through pivotal metrics such as sprint completion times, defect rates, and adherence to project deadlines. By employing simulations and load testing, researchers can evaluate the system's efficiency and effectiveness as project sizes expand [60,61].

Another significant avenue for investigation is the enhancement of models for human–AI collaboration. Developing pioneering interfaces and mechanisms for mutual learning between human developers and AI agents can substantially enhance team dynamics. Refining MAS architectures to align more effectively with Agile practices may enable LLMs to simulate and facilitate human-like interactions within software development teams [62,63]. This includes embedding predictive analytics into MAS to provide actionable insights into potential project delays, allowing teams to implement corrective measures proactively.

Moreover, practical validation of the proposed ecosystem remains essential. Deploying the system across real-world software projects will facilitate the evaluation of its impact on productivity, quality, and team dynamics. Integration testing to ensure compatibility with tools such as Continuous Integration/Continuous Deployment (CI/CD) pipelines and issue trackers is crucial for its seamless incorporation into existing development workflows [61]. Empirical studies in this context will generate valuable feedback for refining the system and addressing implementation challenges.

8.2. Technological Advancements

Emerging technologies present numerous opportunities to further enhance the proposed framework. Continuous advancements in AI and LLMs necessitate ongoing updates and retraining to sustain system effectiveness and adherence to ethical standards [64]. Investigating innovative MAS architectures capable of dynamically adapting to complex project requirements may lead to more resilient and flexible systems [63,65]. Incorporating sophisticated natural language processing capabilities can enhance the contextual understanding and emotional intelligence of cognitive agents, enabling them to better interpret human inputs and respond effectively across diverse scenarios.

Advancements in cloud computing and distributed systems also offer robust support for the scalability and efficiency of MASs. By leveraging cloud services, such as Amazon Web Services, automation of tasks including load testing, cost management, and service health monitoring can be achieved, thereby improving the system's ability to manage tasks and mitigate risks [61]. Additionally, integrating emerging technologies like blockchain for secure data management [66] and the Internet of Things (IoT) for data collection and analysis can broaden the applicability and robustness of the framework.

By staying attuned to these technological advancements, the framework can evolve to address the changing demands of software engineering. Such evolution is crucial for maintaining relevance and effectiveness, ensuring that the system remains capable of addressing dynamic challenges in project management and software development.

8.3. Security and Privacy Considerations

As AI technologies become integral to project management, addressing ethical considerations is crucial, particularly regarding data privacy, security, and fairness. Protecting sensitive information requires robust encryption methods, stringent access controls, and adherence to data protection regulations such as GDPR. Transparency in AI-driven decision-making processes further fosters trust among team members and stakeholders. Bias mitigation in AI models is another critical challenge, addressed through continuous monitoring of outputs to identify and rectify biases caused by imbalanced training data or flawed algorithms. Employing fairness-aware machine learning techniques and promoting diversity in training datasets are effective strategies to reduce bias. Furthermore, fostering an organizational culture that prioritizes ethical AI usage through training and awareness programs supports the responsible and trustworthy integration of AI technologies [64].

Establishing clear guidelines and providing training for both human team members and AI agents can nurture a collaborative working environment. This includes stressing accountability, encouraging responsible AI usage, and facilitating open discussions about ethical dilemmas that may arise during project development [67].

However, LLMs may still inadvertently introduce biases or misinterpret nuanced project requirements, particularly when dealing with ambiguous instructions or culturally specific contexts. Their knowledge is derived from training data that can reflect historical imbalances or inaccuracies, potentially affecting the prioritization and communication of tasks. Recognizing these limitations is essential to ensure that human oversight and continuous refinement of the model remain central aspects of a reliable and equitable software project management ecosystem [68].

8.4. Integrating Cognitive Agents into the Enterprise-Wide Agile Scaling Framework

The Scaled Agile Framework is widely adopted for scaling Agile practices in large organizations, providing a structured approach through its core conceptual layers, which collectively enable organizations to achieve business agility [25]. As illustrated in Figure 13, these layers emphasize the progression from organizational strategy down to team-level execution and highlight how cognitive agents and MASs can support each level.

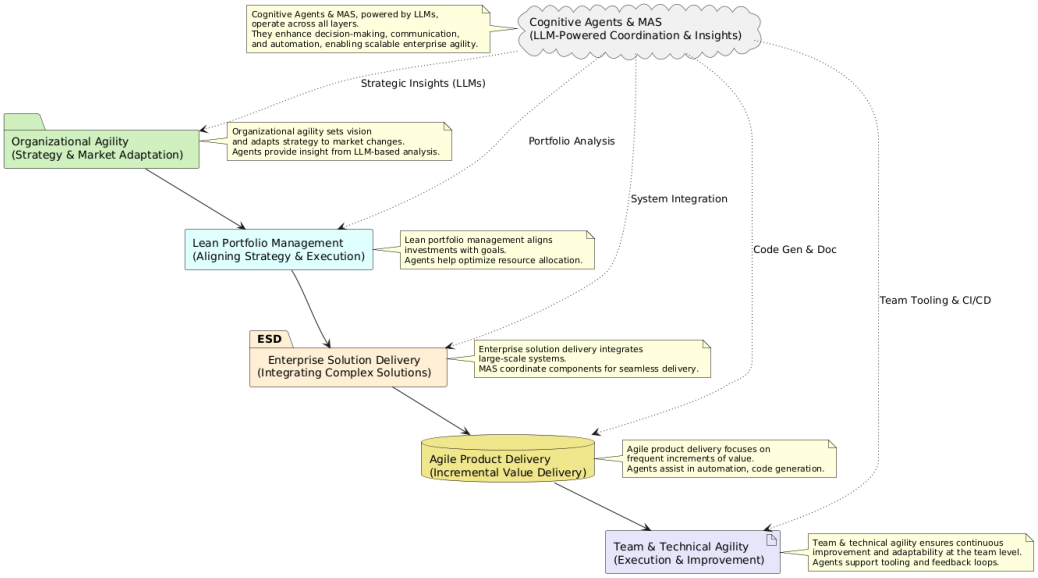

Figure 13. Conceptual enterprise-scale Agile layers with cognitive agents and MASs (inspired by principles in frameworks such as SAFe 6.0 [25,69]).

By leveraging cognitive agents and Multi-Agent Systems at these higher conceptual layers, organizations can enhance decision-making, streamline operations, and foster continuous alignment of business strategies. For example, cognitive agents can automate data analysis to support strategic planning, ensuring that initiatives remain aligned with shifting market demands and organizational priorities. Additionally, MASs can optimize resource allocation by providing real-time insights, enabling more adaptive budgeting and portfolio management.

Future work involves exploring how the integration of cognitive agents and Large Language Models can further enhance enterprise-level Agile frameworks. By embedding cognitive agents into these layers, organizations can automate coordination tasks, improve communication efficiency, and facilitate decision-making processes. For instance, cognitive agents could assist in strategic alignment through predictive analytics, or in Agile Product Delivery by automating code generation and documentation [36,37].

Moreover, incorporating Multi-Agent Systems powered by LLMs can support cross-team collaboration within scaled Agile initiatives. These agents can simulate human-like interactions, enabling more effective coordination across teams and enhancing the scalability of Agile practices [8].

Investigating the challenges and opportunities of integrating cognitive agents into enterprise-level Agile frameworks is a promising area for future research. This includes assessing the impact on team dynamics, addressing potential ethical concerns, and developing best practices for implementation.

9. Conclusions and Summary

The conclusions drawn from this study emphasize the transformative potential of integrating cognitive agents, Multi-Agent Systems, and Large Language Models into Agile development frameworks. By exploring the synergy between cognitive and autonomous systems, this research provides a comprehensive approach to enhancing project management processes and achieving improved productivity and collaboration. The findings underscore the importance of leveraging advanced technologies to address contemporary challenges in software engineering, ensuring adaptability and scalability in complex project environments.

9.1. Summary of Contributions

This study has demonstrated the substantial advancements achieved through the integration of cognitive agents, Multi-Agent Systems, and Large Language Models into Agile development frameworks such as SAFe and SCRUM. Rather than making definitive claims about complex task automation, decision-making improvements, or productivity gains, we now present these as preliminary high-level opportunities indicated by our initial simulations. For example, the framework showed potential in automating routine backlog item refinements and code snippet generation, offered preliminary indications of more autonomous architectural recommendations, and hinted at reduced turnaround times for certain project tasks. However, due to space and scope constraints, rigorous quantification of these gains or direct comparisons against human-led baselines are deferred to future research.

The research introduced a cohesive framework that facilitates improved task management and fosters collaboration among team members. The utilization of cognitive agents has proven effective in supporting continuous adaptation to evolving project needs, aligning seamlessly with the principles of Agile methodologies. Concept diagrams provided in Figure 14 illustrate how the MAS interacts with critical project management components and external systems. These diagrams emphasize the system's ability to theoretically automate task allocation, problem-solving, collaboration, and documentation, while ensuring continuous learning, adaptability, and transparency within the system. Specific empirical validation of these capabilities will be addressed in future studies.

Additionally, the practical applications of MASs and LLMs extend beyond software development and project management into domains such as healthcare, education, and financial modeling. These systems streamline workflows, reduce the cognitive burden on human teams, and improve communication between developers, clients, and stakeholders [63,67]. By addressing common Agile challenges, including communication gaps and scaling difficulties, the research provides valuable insights into improving Agile project outcomes.

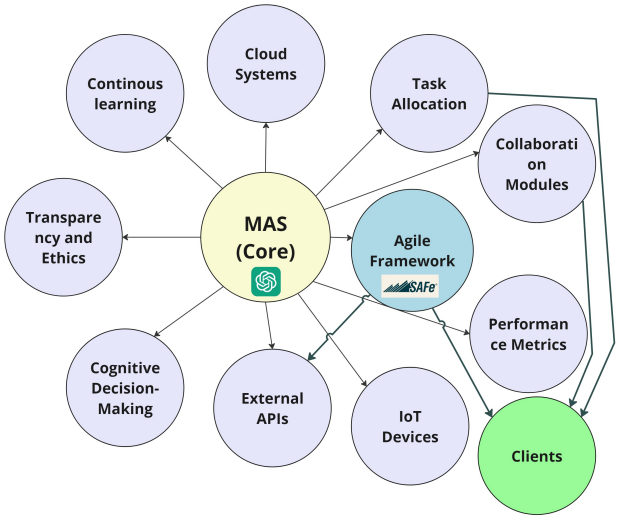

Figure 14. MAS concept diagram.

9.2. Implications for Practice

The integration of these technologies presents significant implications for Agile software project management. Organizations that adopt MASs and LLMs can realize strategic benefits, including enhanced decision-making capabilities, improved risk management, and better project insights. Automating routine tasks enables human team members to focus on strategic project aspects, thereby boosting team productivity and fostering innovation.

Furthermore, the ability of these systems to handle advanced projects with greater precision and reduced operational costs can confer a competitive advantage in the software industry. By ensuring compatibility with existing cloud platforms and software libraries, the proposed architectures enable seamless incorporation into development practices [61]. This adaptability is critical in software engineering, where flexibility and responsiveness to change are essential for success.

9.3. Final Remarks

In conclusion, the integration of cognitive agents, MASs, and LLMs into Agile development frameworks signifies a pivotal advancement in software engineering. While the potential benefits are considerable, it is essential to address the challenges associated with these technologies, particularly concerning data privacy, security, and transparency in AI-driven decisions [64]. Future research should prioritize refining human–AI collaboration models, enhancing agent capabilities, and ensuring that ethical considerations remain central to technological progress.

The ongoing evolution of these systems will undoubtedly shape the future of software engineering, enabling organizations to manage complex projects more effectively and efficiently. By embracing these innovations, the industry can anticipate improved decision-making processes, optimized workflows, and dynamic adaptation to project needs, driving progress and achieving success in an increasingly competitive landscape.

Author Contributions: Conceptualization, K.C., J.A.C. and E.N.-S.; Methodology, K.C. and J.A.C.; Software, K.C.; Validation, K.C.; Formal analysis, J.A.C. and E.N.-S.; Investigation, K.C. and J.A.C.; Data curation, K.C.; Writing—original draft, K.C. and J.A.C.; Writing—review & editing, E.N.-S.; Visualization, K.C.; Supervision, J.A.C.; Project administration, E.N.-S. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Abrahamsson, P.; Salo, O.; Ronkainen, J.; Warsta, J. Agile Software Development Methods: Review and Analysis. *arXiv* **2017**, arXiv:1709.08439.
2. Perkusich, M.; Chaves E Silva, L.; Costa, A.; Ramos, F.; Saraiva, R.; Freire, A.; Dilorenzo, E.; Dantas, E.; Santos, D.; Gorgônio, K.; et al. Intelligent software engineering in the context of agile software development: A systematic literature review. *Inf. Softw. Technol.* **2020**, *119*, 106241. [CrossRef]
3. Dingsøyr, T.; Nerur, S.; Balijepally, V.; Moe, N.B. A decade of agile methodologies: Towards explaining agile software development. *J. Syst. Softw.* **2012**, *85*, 1213–1221. [CrossRef]
4. Shastri, Y.; Hoda, R.; Amor, R. The role of the project manager in agile software development projects. *J. Syst. Softw.* **2021**, *173*, 110871. [CrossRef]
5. Pressman, R.S.; Maxim, B.R. *Software Engineering: A Practitioner's Approach*, 9th ed.; McGraw-Hill Education: New York, NY, USA, 2020.
6. Rubin, K.S.; Cohn, M.; Jeffries, R. *Essential Scrum: A Practical Guide to the Most Popular Agile Process*; The Addison-Wesley Signature Series; Addison-Wesley: Upper Saddle River, NJ, USA; Boston, MA, USA; Indianapolis, Indiana; San Francisco, CA, USA; New York, NY, USA; Toronto, ON, Canada; Montreal, QC, Canada; London, UK; Munich, Germany; Paris, France; Madrid, Spain; Capetown, South Africa; Sydney, Australia; Tokyo, Japan; Singapore; Mexico City, Mexico, 2013.
7. Scrum.org. Scrum Framework. Scrum.org. 2020. Available online: https://www.scrum.org (accessed on 24 December 2024).
8. Cruz, C.J.X. Transforming Competition into Collaboration: The Revolutionary Role of Multi-Agent Systems and Language Models in Modern Organizations. *arXiv* **2024**, arXiv:2403.07769. [CrossRef]
9. Spanoudakis, N.I. Engineering Multi-agent Systems with Statecharts: Theory and Practice. *SN Comput. Sci.* **2021**, *2*, 317. [CrossRef]
10. Guo, Z.; Jin, R.; Liu, C.; Huang, Y.; Shi, D.; Supryadi.; Yu, L.; Liu, Y.; Li, J.; Xiong, B.; et al. Evaluating Large Language Models: A Comprehensive Survey. *arXiv* **2023**, arXiv:2310.19736.
11. Barua, S. Exploring Autonomous Agents through the Lens of Large Language Models: A Review. *arXiv* **2024**, arXiv:2404.04442. [CrossRef]
12. Dvivedi, S.S.; Vijay, V.; Pujari, S.L.R.; Lodh, S.; Kumar, D. A Comparative Analysis of Large Language Models for Code Documentation Generation. *arXiv* **2024**, arXiv:2312.10349. [CrossRef]
13. Tian, R.; Ye, Y.; Qin, Y.; Cong, X.; Lin, Y.; Pan, Y.; Wu, Y.; Hui, H.; Liu, W.; Liu, Z.; et al. DebugBench: Evaluating Debugging Capability of Large Language Models. *arXiv* **2024**, arXiv:2401.04621. [CrossRef]
14. Yuan, S.T.; Yokoo, M.; Goos, G.; Hartmanis, J.; Van Leeuwen, J.; Carbonell, J.G.; Siekmann, J. (Eds.) *Intelligent Agents: Specification, Modeling, and Applications: 4th Pacific Rim International Workshop on Multi-Agents, PRIMA 2001 Taipei, Taiwan, 28–29 July 2001 Proceedings*; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2001; Volume 2132. [CrossRef]
15. IEEE Foundation for Intelligent Physical Agents (FIPA). Design process documentation template. In *Manual SC00097B, IEEE FIPA DPDF Working Group, IEEE FIPA*; Status: Standard tex.changelog: (Initial); Cossentino, M., Molesini, A., Omicini, A., Hilaire, V., Fuentes, R., DeLoach, S., Migeon, F., Bonjean, N., Gleizes, M.P., Maurel, C., et al., Eds.; FIPA: Alameda, CA, USA, 2012.
16. Sumers, T.; Yao, S.; Narasimhan, K.; Griffiths, T. Cognitive Architectures for Language Agents. Transactions on Machine Learning Research. In Review. 2024. Available online: https://openreview.net/forum?id=1i6ZCvflQJ (accessed on 24 December 2024).
17. Cinkusz, K.; Chudziak, J.A. Towards LLM-augmented multiagent systems for agile software engineering. In Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, Sacramento, CA, USA, 27 October–1 November 2024; pp. 2476–2477. [CrossRef]
18. He, J.; Treude, C.; Lo, D. LLM-Based Multi-Agent Systems for Software Engineering: Vision and the Road Ahead. *arXiv* **2024**, arXiv:2404.04834. [CrossRef]
19. Feng, P.; He, Y.; Huang, G.; Lin, Y.; Zhang, H.; Zhang, Y.; Li, H. AGILE: A Novel Reinforcement Learning Framework of LLM Agents. *arXiv* **2024**, arXiv:2405.14751. [CrossRef]
20. Jin, H.; Huang, L.; Cai, H.; Yan, J.; Li, B.; Chen, H. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future. *arXiv* **2024**, arXiv:2408.02479. [CrossRef]
21. Rasheed, Z.; Sami, M.A.; Kemell, K.K.; Waseem, M.; Saari, M.; Systä, K.; Abrahamsson, P. CodePori: Large-Scale System for Autonomous Software Development Using Multi-Agent Technology. *arXiv* **2024**, arXiv:2402.01411. [CrossRef]
22. Talebirad, Y.; Nadiri, A. Multi-Agent Collaboration: Harnessing the Power of Intelligent LLM Agents. *arXiv* **2023**, arXiv:2306.03314. [CrossRef]

23. Li, H.; Chong, Y.Q.; Stepputtis, S.; Campbell, J.; Hughes, D.; Lewis, M.; Sycara, K. Theory of Mind for Multi-Agent Collaboration via Large Language Models. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Singapore, 6–10 December 2023; pp. 180–192. [CrossRef]
24. Singhal, K.; Azizi, S.; Tu, T.; Mahdavi, S.S.; Wei, J.; Chung, H.W.; Scales, N.; Tanwani, A.; Cole-Lewis, H.; Pfohl, S.; et al. Publisher Correction: Large language models encode clinical knowledge. *Nature* **2023**, *620*, E19. [CrossRef]
25. Scaled Agile, Inc. *SAFe 6.0 Framework*; Scaled Agile, Inc.: Boulder, CO, USA, 2024.
26. Kim, A.G.; Muhn, M.; Nikolaev, V.V. Financial Statement Analysis with Large Language Models. *arXiv* **2024**, arXiv:2407.17866. [CrossRef]
27. Chiang, C.H.; Lee, H.y. Can Large Language Models Be an Alternative to Human Evaluations? In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Toronto, ON, Canada, 9–14 July 2023; pp. 15607–15631. [CrossRef]
28. Rocha, F.G.; Misra, S.; Soares, M.S. Guidelines for Future Agile Methodologies and Architecture Reconciliation for Software-Intensive Systems. *Electronics* **2023**, *12*, 1582. [CrossRef]
29. Highsmith, J.A. *Agile Project Management: Creating Innovative Products*, 2nd ed.; The Agile Software Development Series; Addison-Wesley: Upper Saddle River, NJ, USA, 2010.
30. Schwaber, K.; Sutherland, J. The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game. Available online: https://scrumguides.org/scrum-guide.html (accessed on 24 December 2024).
31. Knaster, R.; Leffingwell, D. *SAFe Distilled: SAFe 5.0: Achieving Business Agility with the Scaled Agile Framework*; Addison-Wesley: Hoboken, NJ, USA, 2020.
32. Ebert, C.; Paasivaara, M. Scaling agile. *IEEE Softw.* **2017**, *34*, 98–103. [CrossRef]
33. Russell, S.J.; Norvig, P. *Artificial Intelligence: A Modern Approach*, 4th ed.; Pearson Series in Artificial Intelligence; Pearson: Hoboken, NJ, USA, 2021.
34. OpenAI.; Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F.L.; Almeida, D.; Altenschmidt, J.; Altman, S.; et al. GPT-4 Technical Report. *arXiv* **2024**, arXiv:2303.08774. [CrossRef]
35. Brown, T.B.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language Models are Few-Shot Learners. *arXiv* **2020**, arXiv:2005.14165.
36. Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H.P.d.O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. Evaluating Large Language Models Trained on Code. *arXiv* **2021**, arXiv:2107.03374.
37. Svyatkovskiy, A.; Deng, S.K.; Fu, S.; Sundaresan, N. IntelliCode compose: Code generation using transformer. In Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual, 17–18 November 2020; pp. 1433–1443. [CrossRef]
38. Weng, L. LLM-Powered Autonomous Agents; Online Resource, 2023. Available online: https://lilianweng.github.io/posts/2023-06-23-agent/ (accessed on 24 December 2024).
39. Hang, C.N.; Wei Tan, C.; Yu, P.D. MCQGen: A large language model-driven MCQ generator for personalized learning. *IEEE Access Pract. Innov. Open Solut.* **2024**, *12*, 102261–102273. [CrossRef]
40. Wooldridge, M.J. *An Introduction to Multiagent Systems*, 2nd ed.; Wiley: Chichester, UK, 2012.
41. Kostka, A.; Chudziak, J.A. Synergizing logical reasoning, long-term memory, and collaborative intelligence in multi-agent LLM systems. In Proceedings of the Pacific Asia Conference on Language, Information and Computation (PACLIC 38), Tokyo, Japan, 7–9 December 2024.
42. Chudziak, J.A.; Wawer, M. ElliottAgents: A natural language-driven multi-agent system for stock market analysis and prediction. In Proceedings of the 38th Pacific Asia Conference on Language, Information and Computation, Tokyo, Japan, 7–9 December 2024. (In Press)
43. Durfee, E.; Lesser, V.; Corkill, D. Trends in cooperative distributed problem solving. *IEEE Trans. Knowl. Data Eng.* **1989**, *1*, 63–83. [CrossRef]
44. Macal, C.M.; North, M.J. Tutorial on agent-based modelling and simulation. *J. Simul.* **2010**, *4*, 151–162. [CrossRef]
45. Younge, A.J.; von Laszewski, G.; Wang, L.; Lopez-Alarcon, S.; Carithers, W. Efficient resource management for Cloud computing environments. In Proceedings of the International Conference on Green Computing, Chicago, IL, USA, 15–18 August 2010; pp. 357–364. [CrossRef]
46. Nguyen, M.H.; Chau, T.P.; Nguyen, P.X.; Bui, N.D.Q. AgileCoder: Dynamic Collaborative Agents for Software Development based on Agile Methodology. *arXiv* **2024**, arXiv:2406.11912.
47. Jennings, N.R. On agent-based software engineering. *Artif. Intell.* **2000**, *117*, 277–296. [CrossRef]
48. Weiss, G. (Ed.) *Multiagent Systems*, 2nd ed.; Intelligent Robotics and Autonomous Agents; The MIT Press: Cambridge, MA, USA; London, UK, 2013.
49. Ferber, J.; Ferber, J. *Multi-Agent Systems: An Introduction to Distributed Artificial Intelligence*, 1st ed.; Addison-Wesley: Boston, MA, USA, 1999.

50. Qian, C.; Liu, W.; Liu, H.; Chen, N.; Dang, Y.; Li, J.; Yang, C.; Chen, W.; Su, Y.; Cong, X.; et al. ChatDev: Communicative Agents for Software Development. *arXiv* **2024**, arXiv:2307.07924.
51. Fenton, N.E.; Bieman, J. *Software Metrics: A Rigorous and Practical Approach*, 3rd ed.; Innovations in Software Engineering and Software Development; CRC Press: Boca Raton, FL, USA, 2015. [CrossRef]
52. Cinkusz, K.; Chudziak, J. Communicative agents for software project management and system development. In Proceedings of the 21th International Conference on Modeling Decisions for Artificial Intelligence MDAI 2024, Tokyo, Japan, 27–31 August 2024; Torra, V., Narukawa, Y., Kikuchi, H., Eds.; ISBN 978-91-531-0238-0
53. Qiao, B.; Li, L.; Zhang, X.; He, S.; Kang, Y.; Zhang, C.; Yang, F.; Dong, H.; Zhang, J.; Wang, L.; et al. TaskWeaver: A Code-First Agent Framework. *arXiv* **2024**, arXiv:2311.17541.
54. Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; Yao, S. Reflexion: Language Agents with Verbal Reinforcement Learning. *arXiv* **2023**, arXiv:2303.11366.
55. LangChain. Version 0.2. Introduction to LangChain. Online Resource. 2023. Available online: https://python.langchain.com/v0.2/docs/introduction/ (accessed on 24 December 2024).
56. LangChain. Version 0.2. LangChain Core API Reference. Online Resource. 2023. Available online: https://python.langchain.com/v0.2/api_reference/core/index.html (accessed on 24 December 2024).
57. Peeperkorn, M.; Kouwenhoven, T.; Brown, D.; Jordanous, A. Is Temperature the Creativity Parameter of Large Language Models? *arXiv* **2024**, arXiv:2405.00492.
58. Mazumder, M.; Banbury, C.; Yao, X.; Karlaš, B.; Rojas, W.G.; Diamos, S.; Diamos, G.; He, L.; Parrish, A.; Kirk, H.R.; et al. DataPerf: Benchmarks for Data-Centric AI Development. *arXiv* **2023**, arXiv:2207.10062.
59. Chudziak, J.; Cegielski, R.W.; Meyer, J. Communication management and its impact on successful IT program. *IADIS Int. J. Comput. Sci. Inf. Syst.* **2008**, *1*, 14–28.
60. Horling, B.; Lesser, V. A survey of multi-agent organizational paradigms. *Knowl. Eng. Rev.* **2004**, *19*, 281–316. [CrossRef]
61. Choinski, M.; Chudziak, J.A. Ontological Learning Assistant for Knowledge Discovery and Data Mining. In Proceedings of the 2009 International Multiconference on Computer Science and Information Technology, Mragowo, Poland, 12–14 October 2009; pp. 147–155. [CrossRef]
62. Cabrero-Daniel, B. AI for Agile development: A Meta-Analysis. *arXiv* **2023**, arXiv:2305.08093.
63. Guo, T.; Chen, X.; Wang, Y.; Chang, R.; Pei, S.; Chawla, N.V.; Wiest, O.; Zhang, X. Large Language Model based Multi-Agents: A Survey of Progress and Challenges. *arXiv* **2024**, arXiv:2402.01680.
64. Tariverdi, A. Trust from Ethical Point of View: Exploring Dynamics Through Multiagent-Driven Cognitive Modeling. *arXiv* **2024**, arXiv:2401.07255.
65. Lin, F.; Kim, D.J.; Chen, T.-H. SOEN-101: Code Generation by Emulating Software Process Models Using Large Language Model Agents. *arXiv* **2024**, arXiv:2403.15852.
66. Dorri, A.; Kanhere, S.S.; Jurdak, R. Multi-Agent Systems: A Survey. *IEEE Access* **2018**, *6*, 28573–28593. [CrossRef]
67. Amirkhani, A.; Barshooi, A.H. Consensus in multi-agent systems: A review. *Artif. Intell. Rev.* **2022**, *55*, 3897–3935. [CrossRef]
68. Echterhoff, J.M.; Liu, Y.; Alessa, A.; McAuley, J.; He, Z. Cognitive bias in decision-making with LLMs. In Proceedings of the Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, FL, USA, 12–16 November 2024; Al-Onaizan, Y., Bansal, M., Chen, Y.N., Eds.; Association for Computational Linguistics: Stroudsburg, PA, USA, 2024; pp. 12640–12653. [CrossRef]
69. Scaled Agile, Inc. SAFe Scrum. Online Resource. 2024. Available online: https://scaledagileframework.com/safe-scrum/ (accessed on 24 December 2024).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.